

Sistema de Gestão de Solicitações de Documentos Acadêmicos

Passo a passo para executar o projeto do zero — com e sem Docker — e navegar pelas telas.

Java 21 · Spring Boot 3.5 · PostgreSQL 17 · React 19 + Vite · JWT · JasperReports · Swagger

Este guia é a versão **ilustrada e resumida** do `README.md`. Os quadros pontilhados verdes são espaços reservados para os **prints do sistema**: salve cada imagem na pasta `docs/imagens/` com o nome indicado na legenda e gere o PDF novamente — a imagem entra sozinha no lugar do quadro.

1. Pré-requisitos

Cenário	Você precisa de	Conferir com
Com Docker (recomendado)	Docker Desktop 24+ (inclui Docker Compose) · Node.js 20+ e npm (para o frontend) · Git	<code>docker --version</code> <code>node --version</code>
Sem Docker	JDK 21 · PostgreSQL 17 · Node.js 20+ e npm · Git (o Maven não precisa ser instalado — o projeto traz o <code>mvnw</code>)	<code>java -version</code> <code>psql --version</code>

Dica: no modo com Docker você **não** precisa de Java nem de PostgreSQL instalados na máquina — o container cuida disso. O Node continua sendo necessário porque o frontend roda fora do container.

1 Clonar o projeto

```
git clone <url-do-repositorio> documentos-academicos
cd documentos-academicos
```

2 Configurar as variáveis de ambiente

Há dois arquivos `.env`: um para o backend (raiz) e um para o frontend.

```
# backend (na raiz do projeto)
cp .env.example .env

# frontend
cp frontend/.env.example frontend/.env
```

Importante: execute o comando `cp` — não copie/cole o texto do exemplo dentro do arquivo. Em seguida edite o `.env` da raiz e defina no mínimo:

- `DB_PASSWORD` — senha do banco;
- `JWT_SECRET` — chave com **32 caracteres ou mais** (sem ela a API nem sobe);
- `ADMIN_PASSWORD` — senha do usuário administrador criado no primeiro start (sem ela o sistema sobe sem nenhum usuário).

3 Subir a aplicação

Escolha **A** (com Docker) **ou B** (sem Docker). Os dois deixam a API em `http://localhost:8080`.

Opção A — com Docker (recomendado)

```
docker compose up --build -d

# confirme que subiu (deve responder {"status":"UP"}):
curl http://localhost:8080/actuator/health

# acompanhar logs / derrubar / apagar o banco:
docker compose logs -f
docker compose down
docker compose down -v
```

Opção B — sem Docker

Crie o banco no PostgreSQL uma única vez:

```
CREATE DATABASE documentos_academicos;
CREATE USER doc_user WITH PASSWORD 'a-mesma-do-DB-PASSWORD';
GRANT ALL PRIVILEGES ON DATABASE documentos_academicos TO doc_user;
```

O Spring Boot não lê o `.env` sozinho. Antes de subir, carregue as variáveis no terminal:

```
# Git Bash / Linux / macOS
set -a; source .env; set +a

# PowerShell
Get-Content .env | ForEach-Object {
```

```
if ($? -match '^s*([^#]=)(.*)$') { [Environment]::SetEnvironmentVariable($matches[1]  
}
```

```
./mvnw spring-boot:run
```

Frontend (igual nos dois modos)

```
cd frontend  
npm install  
npm run dev          # abre em http://localhost:3000
```

4 Usando o sistema

Abra <http://localhost:3000> e entre com o `ADMIN_LOGIN` (padrão `administrador`) e o `ADMIN_PASSWORD` que você definiu.

 Espaço para o screenshot

Figura 1 — Tela de login `docs/imagens/01-login.png`

Roteiro sugerido de primeiro acesso:

1. No menu **Cadastros**, crie ao menos um **aluno**, um **curso** e um **tipo de documento**.
2. Em **Solicitações** → **Nova**, monte a primeira solicitação usando os botões/chips de seleção.
3. Movimente o status (Em análise → Aprovada → Emitida) e acompanhe o fluxo.
4. Gere o **relatório PDF** pela listagem.

 Espaço para o screenshot

Figura 2 — Dashboard com os indicadores `docs/imagens/02-dashboard.png`

 Espaço para o screenshot

Figura 3 — Formulário de nova solicitação (chips interativos) `docs/imagens/03-nova-solicitacao.png`

 Espaço para o screenshot

Figura 4 — Listagem de solicitações [docs/imagens/04-listagem.png](#)

 Espaço para o screenshot

Figura 5 — Modal do fluxo (BPMN) destacando a etapa atual [docs/imagens/05-fluxo-bpmn.png](#)

 Espaço para o screenshot

Figura 6 — Detalhe da solicitação com anexos [docs/imagens/06-detalhe.png](#)

 Espaço para o screenshot

Figura 7 — Relatório PDF gerado [docs/imagens/07-relatorio-pdf.png](#)

 Espaço para o screenshot

Figura 8 — Tela de cadastros básicos [docs/imagens/08-cadastros.png](#)



 **Espaço para o screenshot**

Figura 9 — Aplicação no modo escuro [docs/imagens/09-tema-escuro.png](#)

5 Documentação interativa (Swagger)

Com o backend rodando, o Swagger UI fica em:

Recurso	Endereço
Swagger UI	<code>http://localhost:8080/swagger-ui.html</code>
OpenAPI (JSON)	<code>http://localhost:8080/v3/api-docs</code>

Como testar um endpoint protegido:

1. Abra o Swagger e chame `POST /api/auth/login` com `login` e `senha` do admin — copie o `token` da resposta.
2. Clique em **Authorize** (cadeado, canto superior direito) e cole o token. Não precisa escrever "Bearer" — o esquema já é *bearer*.
3. Agora todos os endpoints usam o token; teste, por exemplo, `GET /api/solicitacoes`.

Observação: o Swagger fica **desligado no perfil `prod`** (é uma superfície de ataque desnecessária em produção). Ele está disponível nos perfis `dev` e `docker`.

📷 Espaço para o screenshot

Figura 10 — Swagger UI `docs/imagens/10-swagger.png`

📷 Espaço para o screenshot

Figura 11 — Botão Authorize com o token JWT `docs/imagens/11-swagger-authorize.png`

6. Rodar os testes

```
# unitários (H2, rápido) - 104 testes
./mvnw test
```

```
# unitários + integração (Testcontainers, exige Docker) + análise estática
./mvnw verify

# frontend
cd frontend && npm run build && npm run lint
```

7. Fluxo de status (resumo)

ABERTA → **EM_ANÁLISE** → (**APROVADA** → **EMITIDA**) ou **REPROVADA**. As transições são validadas pelo servidor (o enum define o que é permitido); a interface só facilita com o modal BPMN. Consulte o [README.md](#) para a tabela completa de permissões e regras.

Gerado a partir de [docs/GUIA-INSTALACAO-E-USO.html](#) · A referência completa está no [README.md](#) do repositório.